Machine Learning Environment Setup: Complete Classroom Notes

1. Introduction to Machine Learning Tools

Before starting any end-to-end Machine Learning (ML) project, a student must know how to set up the workspace. There are three primary ways to work in Data Science:

1. **Local Environment:** Installing software on your own computer (Anaconda).
2. **Kaggle Notebooks:** A cloud-based platform for datasets and coding.
3. **Google Colab:** A Google-based cloud platform, famous for providing free GPU power.

2. Anaconda: The Local Setup

Anaconda is the most popular platform for Data Science.

Why use Anaconda?

Normally, Python is plain. For Data Science, you need many libraries like **NumPy**, **Pandas**, and **Scikit-Learn**. Installing them one by one is difficult. Anaconda "bundles" all these tools into one package.

Installation Steps:

1. Go to anaconda.com.
2. Navigate to **Products** -> **Individual Edition**.
3. Download the installer for your Operating System (Windows, Mac, or Linux).
4. **Note:** It is a large file (~450MB - 500MB).
5. **Installation Tip:** Keep all settings as "Default" (Just click Next, Next, and Finish).

3. Jupyter Notebook: Your Coding Canvas

Once Anaconda is installed, you get a tool called **Jupyter Notebook**. While Anaconda also provides "Spyder" (an IDE), most Data Scientists prefer Jupyter because it is interactive and great for showing data.

How to use Jupyter Notebook:

1. Search for "Jupyter Notebook" in your Start Menu and open it.
2. A black console window will open (don't close it!), and then your web browser will open the Jupyter interface.
3. **Organization:** Always create a new folder for your projects (e.g., "100 Days of ML").
4. **Creating a file:** Click "New" -> "Python 3".

The Two Types of Cells:

- **Code Cell:** Where you write Python. Run it using Shift + Enter.
- **Markdown Cell:** Where you write notes/documentation.

Code Example: Writing your first code

```
print("Hello World")
```

Line-by-line Explanation:

- print(): This is a built-in Python function used to display output.
- "Hello World": This is the text (string) we want to show on the screen.

Markdown Tips (Teacher's Secret):

You can make your notes look like a professional website inside Jupyter:

- Use # for a Big Heading.
- Use ## for a Smaller Heading.
- You can even use **HTML**!
    - Example: Warning will show the word "Warning" in red.

4. Virtual Environments: The "Box" Concept

This is a very important concept for professional developers.

What is a Virtual Environment?

When you install Anaconda, you are in the **"Base"** environment. It has 100+ libraries.

- **The Problem:** If you build a small project that only needs 2 libraries and you upload it to a server, the server will try to install all 100+ libraries from your base environment. This makes your app huge and slow.
- **The Solution:** Create a "Virtual Environment" (think of it as a small, empty box). You only put the libraries you need for *that specific project* inside that box.

Commands for Virtual Environments (Run in Anaconda Prompt):

1. **Create:** conda create --name my_env
2. **Activate:** conda activate my_env
3. **Install tools:** conda install jupyter (You must install Jupyter inside the new environment to use it there).
4. **Delete Environment:** conda remove --name my_env --all

5. Working with Data (CSV Files)

To do Machine Learning, you need data. The most common format is .csv.

How to load a dataset in Jupyter:

1. Download a dataset (from Kaggle).
2. Upload the file into your Jupyter folder.
3. Use the **Pandas** library to read it.

Code Example:

```
import pandas as pd
df = pd.read_csv('employees.csv')
df.head()
```

Line-by-line Explanation:

- import pandas as pd: We bring in the Pandas library and give it a short name pd so it's easier to type.
- df = pd.read_csv('employees.csv'): We use the read_csv function to open the file. We store the data in a variable called df (short for Data Frame).
- df.head(): This command shows the first 5 rows of your data so you can see if it loaded correctly.

6. Cloud Options: Kaggle & Google Colab

Kaggle Notebooks:

- **Pro:** You don't need to install anything on your computer.
- **Portfolio:** Your notebooks are saved on your Kaggle profile. If you make them public, other people can see your work, which helps in getting jobs.

Google Colab:

- **The Power of GPU:** If you are doing "Deep Learning" (working with images or heavy data), your laptop might get very hot and slow. Google Colab gives you a **GPU (Graphics Processing Unit)** for free.
- **How to turn on GPU:** Go to Runtime -> Change runtime type -> Select GPU.

7. Professional Trick: Connecting Kaggle to Colab

Scenario: You find a 2GB dataset on Kaggle.

- *Slow way:* Download 2GB to your PC -> Upload 2GB to Google Colab. (Waste of time/data).
- *Fast way:* Use the Kaggle API to send data directly from Kaggle's server to Google's server.

The Steps:

1. Go to Kaggle Settings -> Click **"Create New API Token"**. This downloads a kaggle.json file.
2. Upload kaggle.json to your Google Colab.

3. Run the API command provided by Kaggle.

Code Example (Downloading and Unzipping):

```
!kaggle datasets download -d developer/dataset-name
!unzip dataset-name.zip
```

Line-by-line Explanation:

- !kaggle datasets download...: The ! tells Colab this is a system command, not Python. This fetches the data directly.
- !unzip...: Datasets usually come as a .zip file. This command extracts all the files (like images or CSVs) so you can use them.

Key Takeaways & Tips

- **Always use Virtual Environments** for real projects to keep them lightweight.
- **Shift + Enter** is the shortcut to run cells in both Jupyter and Colab.
- **Use GPU** in Colab only when working with Deep Learning (Neural Networks). For simple tables (Excel/CSV), CPU is enough.
- **Documentation is key:** Use Markdown cells to explain "Why" you are writing a specific piece of code. This makes your work readable for teammates.